How FT8 Really Works

A plain-language guide to the signal, the 15-second cycle, the error-correcting code, and the arithmetic behind decoding at −20 dB

Terry Gilton, PhD, W7TLG

October 2026

Abstract. FT8 lets a modest station on a noisy HF band exchange callsigns, grid squares and signal reports when the signal is far too weak to hear. The result looks like magic, but it is the product of a handful of ordinary engineering ideas applied with unusual care: compress the message to 77 bits, add a checksum, wrap the result in a very strong error-correcting code, send it as slow tones so that the receiver can use a very narrow filter, and add a synchronization pattern that lets the decoder find a signal it cannot see. This paper explains each idea in turn, starting with what you actually see in the spectrum and on the waterfall, and finishing with the question every thoughtful operator eventually asks: how can a signal 20 dB *below the noise* be decoded without violating Shannon's theorem? (It does not violate it, and the numbers show how close FT8 comes to the limit.) The main text needs no mathematics. Boxed "Going deeper" sections give the details for readers who want them. Every figure was generated by a working Python implementation of the FT8 encoder and a simple receiver, validated against an independent open-source implementation.

Contents

1. The big picture: a 77-bit telegram sent as 79 tones
2. What the signal actually is
3. What you see on the screen
4. The 15-second cycle
5. Fitting a message into 77 bits
6. Protecting the message: the CRC
7. Low-density parity-check coding, without the pain
8. Synchronization: the Costas array
9. Signal-to-noise ratio, "below the noise," and Shannon
10. Practical consequences for the operator
11. Summary
Appendix A. A message, bit by bit
Appendix B. Belief propagation by hand
Appendix C. The link-budget arithmetic
Appendix D. Glossary
Appendix E. How the figures were made
References

How to read this paper

FT8 was released in June 2017 by Nobel prize recipient Joe Taylor, K1JT, and Steve Franke, K9AN, as part of the WSJT-X software package, and it has since become the most heavily used mode on the HF amateur bands. The designers have documented it thoroughly in the QEX article "The FT4 and FT8 Communication Protocols" (Franke, Somerville and Taylor, 2020). That article is written for engineers. This paper has a different aim: to give a working radio amateur a correct mental model of what is on the screen and what the software is doing, without requiring a communications-theory background.

The paper is organized so that you can stop at any point and still have a coherent picture.

- Sections 1 to 3 describe the signal itself and what you see in the spectrum and on the waterfall.
- Section 4 describes the 15-second cycle and the clock.
- Sections 5 to 7 follow a message from keyboard to antenna: packing it into 77 bits, adding the checksum, and the error-correcting code.
- Section 8 explains the Costas synchronization array.
- Section 9 takes on signal-to-noise ratio, "decoding below the noise," and Shannon.
- Section 10 turns the theory into practical operating habits.
- Four appendices give a bit-by-bit worked example, a hand-calculable decoding example, the link-budget arithmetic, and a glossary.

Shaded boxes marked **Going deeper** contain the quantitative details. They can be skipped without losing the thread.

About the figures and numbers. To avoid illustrating FT8 with cartoons, I wrote a small FT8 transmitter and receiver in Python and made the figures from real signals. The transmitter was checked against the open-source `ft8_lib` implementation: the 79-tone sequences were identical for all 13 test messages (standard, reply, report, RR73, 73, CQ-with-modifier, /R suffix and free text), the synthesized audio matched the reference waveform to a correlation of 0.999999, and the reference decoder correctly decodes my audio. The receiver is deliberately simple (no ordered-statistics fallback, no a-priori decoding, no signal subtraction), so its sensitivity is a little worse than WSJT-X; where I quote WSJT-X performance, the number is taken from the published paper and is labeled as such.

1. The big picture: a 77-bit telegram sent as 79 tones

Before looking at any detail, it helps to see the whole chain in one place. When you click "Enable Tx," the following happens, in this order.

1. **Source coding.** Your message (for example `CQ W7TLG DN13`) is converted into exactly **77 bits**. Callsigns and grid squares are not sent as text. They are sent as index numbers into a very large, cleverly organized table, which is why so much fits in so few bits.
2. **Error detection.** A **14-bit checksum** (a CRC) is computed from the 77 bits and appended, giving **91 bits**.
3. **Error correction.** The 91 bits are fed through a **low-density parity-check (LDPC) code** that adds **83 parity bits**, producing a **174-bit codeword**. This is the step that lets the receiver repair damaged bits.
4. **Mapping to tones.** The 174 bits are taken three at a time. Each group of three bits selects one of **8 tones**, so the codeword becomes **58 data symbols**.

5. **Synchronization.** Three copies of a fixed 7-tone **Costas array** are inserted (at the beginning, in the middle, and at the end), adding **21 sync symbols**. The transmission is now **79 symbols** long.
6. **Modulation.** Each symbol lasts 0.16 s and is sent as a pure audio tone, with the tones spaced 6.25 Hz apart. The transitions are smoothed with a Gaussian filter so the signal stays narrow. The audio goes to your SSB transmitter. Total duration: 79 × 0.16 s = **12.64 s**.

The receiver runs the chain backwards: find the signal, measure the energy at each of the eight possible tones in each symbol, convert those measurements to confidence values for the 174 bits, let the LDPC decoder repair the errors, check the CRC, and unpack the 77 bits into text.

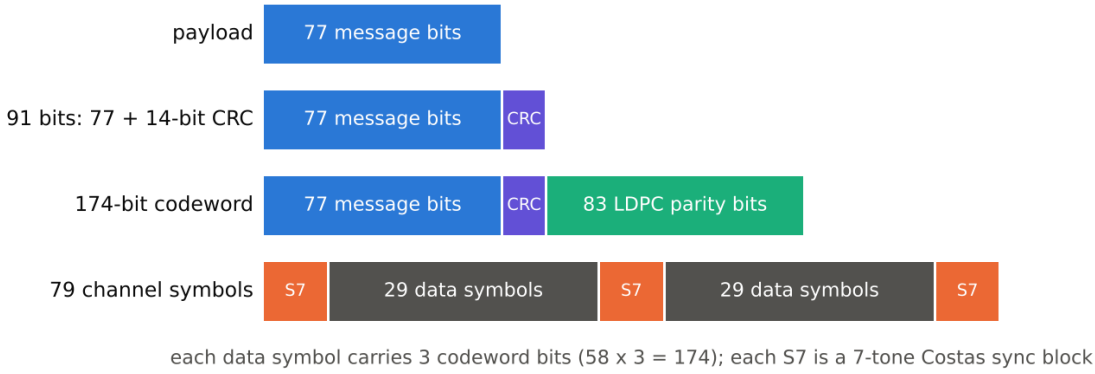


Figure 1. The bit budget. Seventy-seven message bits become 91 with the CRC and 174 with the LDPC parity bits. The 174 bits ride on 58 data symbols, which are interleaved with 21 synchronization symbols to make the 79 transmitted symbols.

The table below summarizes the numbers worth remembering. Each is derived later in the paper.

Quantity	Value
Modulation	8-tone continuous-phase FSK, Gaussian-smoothed (8-GFSK)
Symbol duration	0.160 s (6.25 symbols per second)
Tone spacing	6.25 Hz (equal to the symbol rate)
Occupied bandwidth	about 50 Hz (8 tones × 6.25 Hz)
Symbols per transmission	79 (58 data + 21 synchronization)
Transmission length	12.64 s, starting 0.5 s into a 15 s period
Message payload	77 bits
Checksum	14-bit CRC
Error-correcting code	LDPC (174, 91): 91 information bits, 83 parity bits
Published sensitivity (50 % decode, AWGN)	−20.8 dB in a 2500 Hz reference bandwidth

2. What the signal actually is

2.1 One tone at a time

The most important fact about an FT8 signal is also the simplest: **at any instant, an FT8 transmitter is sending exactly one steady audio tone.** It is not a mixture of many carriers, like OFDM, and it is not a noise-like spread-spectrum signal. It is closer to the oldest digital mode there is (frequency-shift keying, as in RTTY), except that instead of two tones there are eight, and instead of 45 baud it runs at 6.25 baud.

Every 0.16 s the transmitter jumps to one of eight tones. Tone 0 is the lowest and tone 7 is the highest, and neighboring tones are 6.25 Hz apart. If your transmit frequency in the software is set to 1000 Hz, the eight tone frequencies are 1000.00, 1006.25, 1012.50, 1018.75, 1025.00, 1031.25, 1037.50 and 1043.75 Hz. The "frequency" you set is the frequency of tone 0, the bottom of the signal, and the whole signal occupies roughly the 50 Hz above it.

Because the audio goes into an SSB transmitter on upper sideband, those audio tones are translated directly to RF. If your dial frequency is 14.074 MHz and your audio frequency is 1000 Hz, tone 0 is on 14.075000 MHz and tone 7 is on 14.075044 MHz.

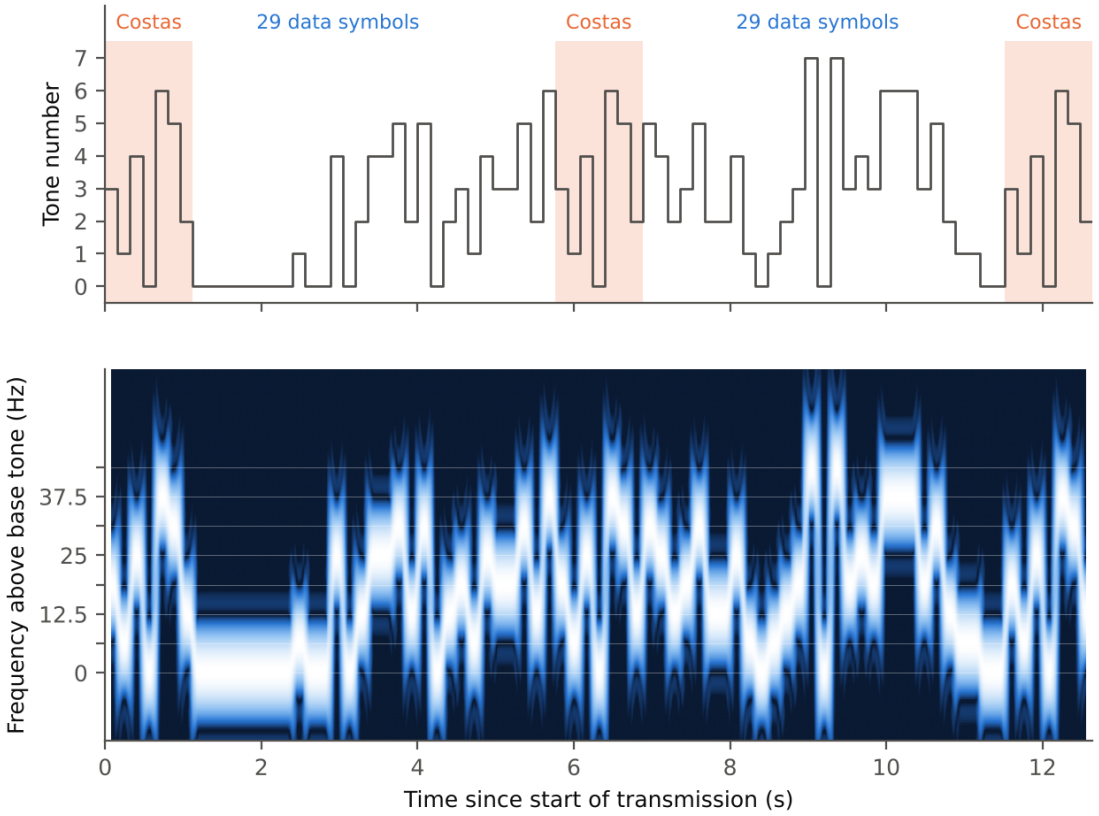


Figure 2. Anatomy of one FT8 transmission of `CQ W7TLG DN13`. Top: the sequence of 79 tone numbers. The three orange bands are the Costas synchronization blocks; the 58 data symbols between them carry the message. Bottom: a spectrogram of the actual synthesized audio. The signal steps among eight frequency positions spaced 6.25 Hz apart, and the whole thing spans only about 50 Hz.

Two consequences follow directly from "one tone at a time."

The envelope is constant. The amplitude of the audio never changes (apart from brief shaping at the very start and end). A constant-envelope signal has the same peak and average power, and it is the easiest kind of signal for a transmitter to send cleanly, because even an amplifier that is slightly nonlinear does not smear a constant-envelope signal across the band the way it smears a multi-tone signal. This is one of the reasons FT8 is forgiving of modest transmitters, though, as Section 10 explains, it is not a license to overdrive the audio.

All the power is concentrated. A 100 W FT8 signal puts all 100 W into a bandwidth of about 6 Hz at any moment. A 100 W SSB voice signal spreads its (peaky, on average much lower) power across 2500 Hz. This concentration of power into a tiny slice of spectrum is the first reason FT8 can be copied when the band is noisy. Receiver noise, though, is distributed across the whole passband, and the receiver can look at the signal through a very narrow window.

2.2 Why 6.25 Hz and 0.16 s?

The tone spacing and the symbol duration are not independent. They are tied together by a piece of signal theory that every radio amateur has met in disguise: the narrower the filter, the longer it takes to respond.

If you want to tell whether a tone is present, you must listen for a while. To resolve two tones that differ by 6.25 Hz, you need to listen for at least 1/6.25 = 0.16 s. If you listen for exactly that long, tones spaced exactly 6.25 Hz apart are perfectly separable ("orthogonal": the energy of one tone contributes exactly zero to the measurement of any other). So FT8 picks the symbol duration $T = 0.16$ s and the tone spacing $1/T = 6.25$ Hz, which gives the narrowest possible spacing that still allows clean separation, and so the narrowest possible signal for a given number of tones.

Going deeper: orthogonality. Over an interval of length T, two complex tones at frequencies f_1 and f_2 have inner product

$$\frac{1}{T}\int_0^T e^{j2\pi f_1 t}\, e^{-j2\pi f_2 t}\, dt = e^{j\pi \Delta f T}\,\frac{\sin(\pi \Delta f T)}{\pi \Delta f T}, \qquad \Delta f = f_1 - f_2.$$

This is zero whenever $\Delta f T$ is a nonzero integer. With $T = 0.16$ s the first zero is at $\Delta f = 6.25$ Hz, so tones on a 6.25 Hz grid do not interfere when each is analyzed over one symbol. This is the "modulation index $h = 1$" of the QEX paper. The receiver's 8-tone detector is simply a bank of eight matched filters, each one 6.25 Hz wide, evaluated once per symbol.

2.3 The Gaussian smoothing

If the transmitter jumped instantly between tones, the abrupt frequency steps would splatter energy far outside the 50 Hz signal. FT8 therefore smooths the frequency transitions with a Gaussian filter (the same family of filter used in GSM mobile phones). The tone does not hop; it glides, rapidly but smoothly, from one value to the next, and the phase of the audio waveform is continuous throughout.

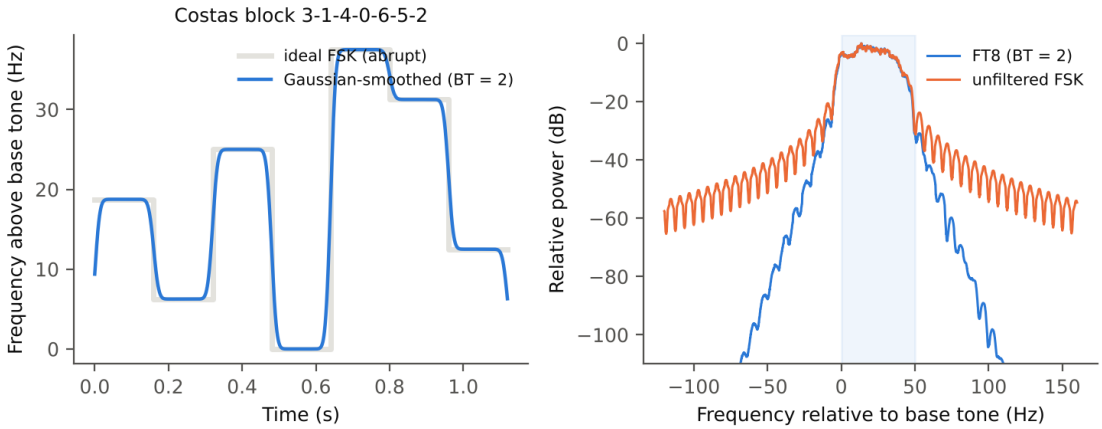


Figure 3. Left: the instantaneous frequency during the Costas block 3-1-4-0-6-5-2, with abrupt keying (gray) and with FT8's Gaussian smoothing (blue). Right: spectra of a full FT8 message with the smoothing (blue) and without it (orange). The smoothing removes the far-out sidebands, so the signal stays inside its 50 Hz slot (shaded).

Going deeper: the smoothing filter. FT8 uses a Gaussian frequency pulse with bandwidth-time product $BT = 2$, where B is the filter's −3 dB bandwidth and T the symbol time. The pulse is the difference of two error functions,

$$g(t) = 1/2\left[\mathrm{erf}(\pi\sqrt{2/\ln 2}\,BT\,(t/T + 1/2)) - \mathrm{erf}(\pi\sqrt{2/\ln 2}\,BT\,(t/T - 1/2))\right],$$

truncated at three symbol lengths. The instantaneous frequency is $f_0 + 1/T\sum_n b_n\; g(t - nT)$, where $b_n \in \{0, \ldots, 7\}$ is the tone number of symbol n, and the phase is the integral of that frequency. Because $BT = 2$ is fairly wide, each symbol is only slightly smeared into its neighbors, which costs very little in detection performance while greatly improving spectral containment. The Python used for the figures implements exactly this expression.

3. What you see on the screen

3.1 The waterfall

When you watch the WSJT-X waterfall (or any SDR waterfall) during FT8, the horizontal axis is audio frequency, brightness is signal strength, and time runs down the screen. A steady carrier would appear as a straight vertical line. An FT8 signal is a **narrow, slightly wiggling vertical streak**, about 50 Hz wide, that appears at the start of a 15-second period, lasts about 12.6 seconds, and then stops. The wiggle is the tone hopping: every 0.16 s the signal steps up or down by some multiple of 6.25 Hz.

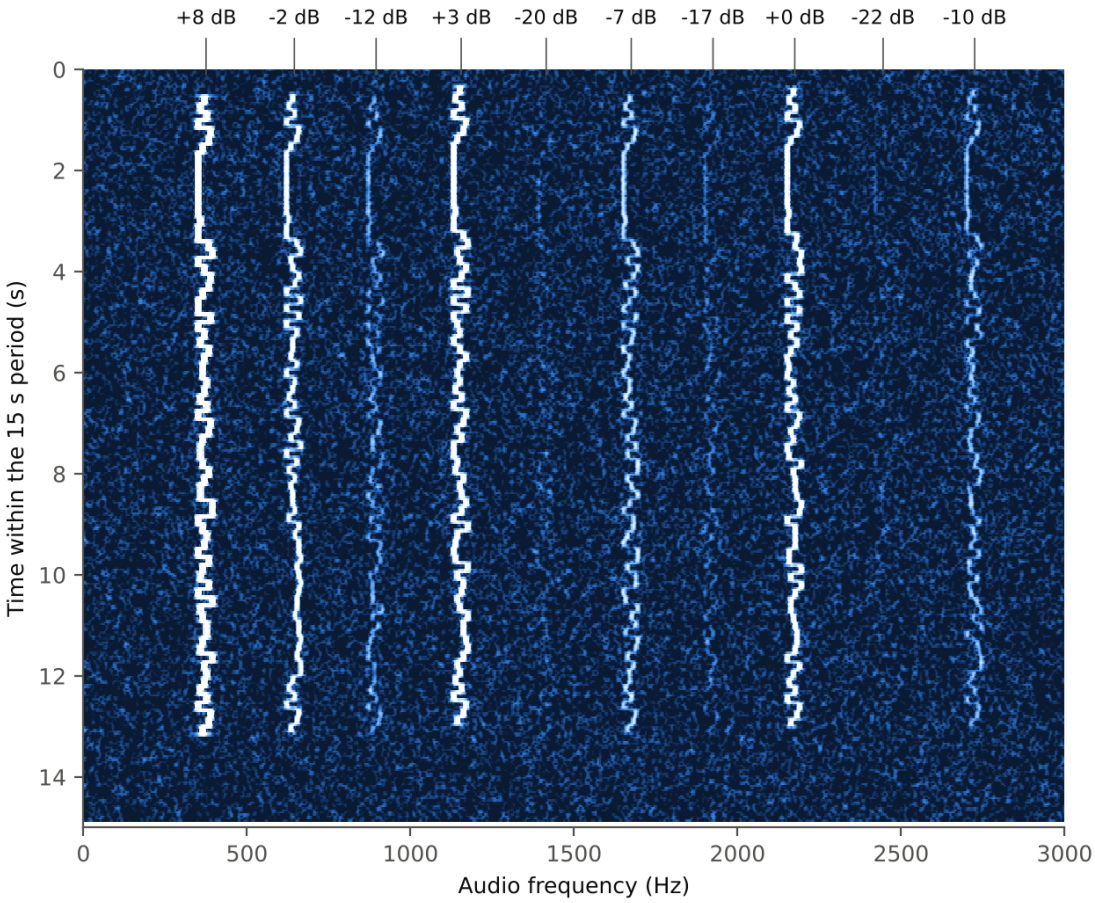


Figure 4. A simulated 3 kHz FT8 passband during one 15-second period, with ten stations at different strengths. The label above each streak is its SNR in the usual 2500 Hz reference bandwidth. The strong signals (+8 to −2 dB) are obvious. The −12 to −17 dB signals are faint but visible. The −20 and −22 dB signals are essentially invisible, yet the decoder can recover them.

A few observations are worth making explicit, because they correct common misunderstandings.

- **Streak brightness does not tell you whether the signal will decode.** A bright signal decodes easily. A signal that you cannot see on the waterfall may still decode, because the decoder integrates evidence from all 79 symbols and uses knowledge of the code, which your eye cannot do.
- **The streaks are 50 Hz wide, and you can fit many of them in a passband.** A 3 kHz passband has room for about 60 non-overlapping signals. In practice, stations may sit as close as 6.25 Hz and overlap, and the decoder still separates many of them, because two signals only collide when they happen to use the same tone in the same symbol.
- **All the transmissions start and stop together.** Everyone who is transmitting in a given period (even or odd) starts within a fraction of a second of everyone else. The gaps at the top and bottom of the waterfall in the figure are the 0.5 s before transmissions start and the roughly 1.9 s after they end, when the decoder is running.
- **The green and red markers in WSJT-X's wide graph** show where your receive frequency and your transmit frequency are. They only tell the software where to decode and where to transmit. The software still decodes the entire passband.

3.2 What the decoder “sees”

The waterfall is a picture designed for people. The decoder works with something different and much more specific. For each of the 79 symbol intervals, it measures how much energy is present at each of the eight possible tone frequencies. The result is a table of 79 columns by 8 rows. If there were no noise, every column would contain exactly one bright cell, the tone that was sent.

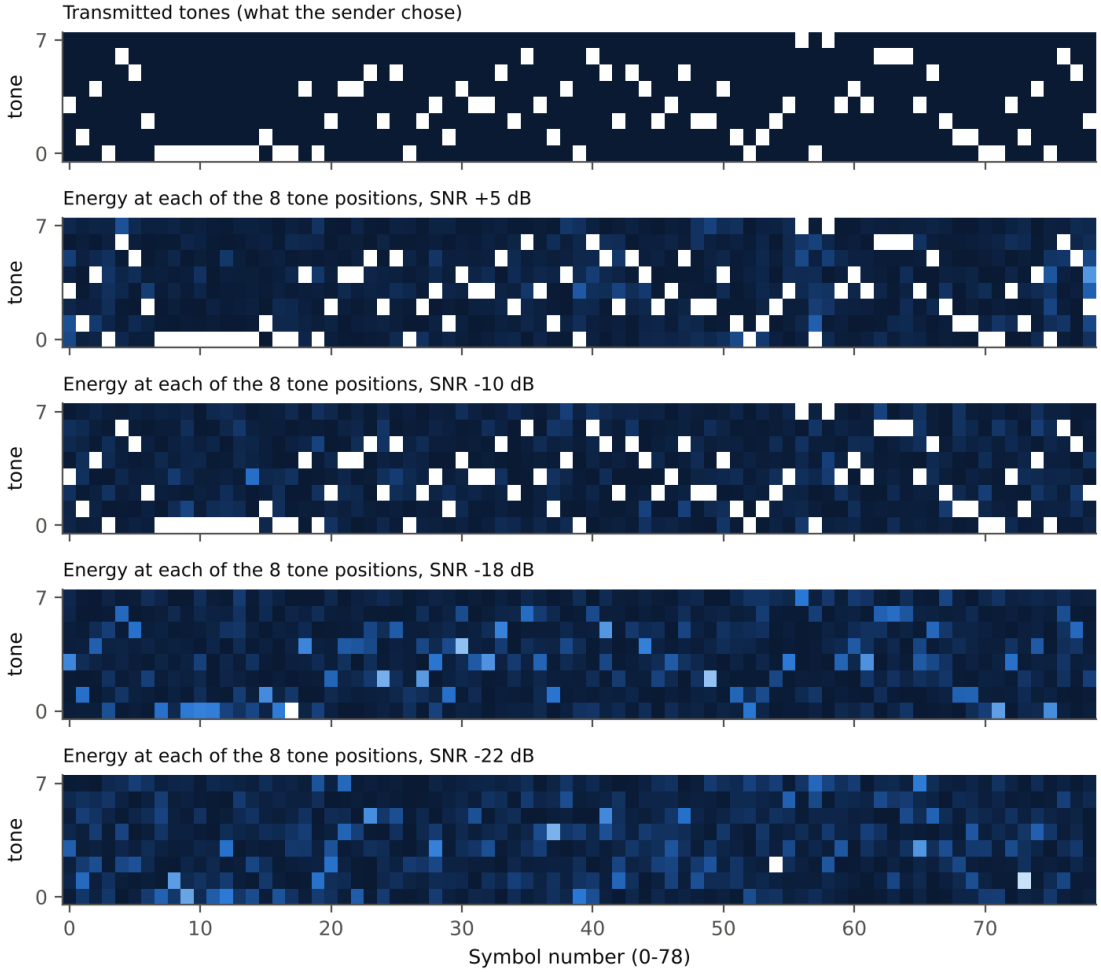


Figure 5. The table of tone energies that the decoder works with (columns are the 79 symbols, rows are the eight tones). Top row: the tones actually sent. Below: the measured energies at four signal-to-noise ratios. At +5 dB the pattern is unmistakable. At −10 dB it is still clear. At −18 dB only the strongest cells stand out, and at −22 dB the pattern is hidden in noise-created bright spots. The decoder succeeds at −18 dB because it checks which candidate messages are *consistent with the whole table*, not because any single column is clear.

This is the key to the whole mode: the decoder is not trying to see individual tones. It is asking, “Given this noisy table, which of the 2^{77} possible messages is most consistent with it?” The error-correcting code (Section 7) is what makes that question answerable.

4. The 15-second cycle

4.1 Timing within a period

FT8 operates in fixed 15-second periods that start at :00, :15, :30 and :45 of every minute. Everyone who is transmitting or receiving uses the same clock. The even periods (:00 and :30) and the odd periods (:15 and :45) are conventionally used by the two stations in a QSO, so one transmits while the other listens.

Inside each 15-second period:

- **0 to 0.5 s.** Quiet. This delay lets the receiving station's decoder finish the previous period's work, and it gives slow clocks some margin.
- **0.5 s to 13.14 s.** The 79-symbol transmission (12.64 s).
- **13.14 s to 15 s.** Quiet again. The receiving software uses roughly this interval to do its final decoding passes, and it displays the results.

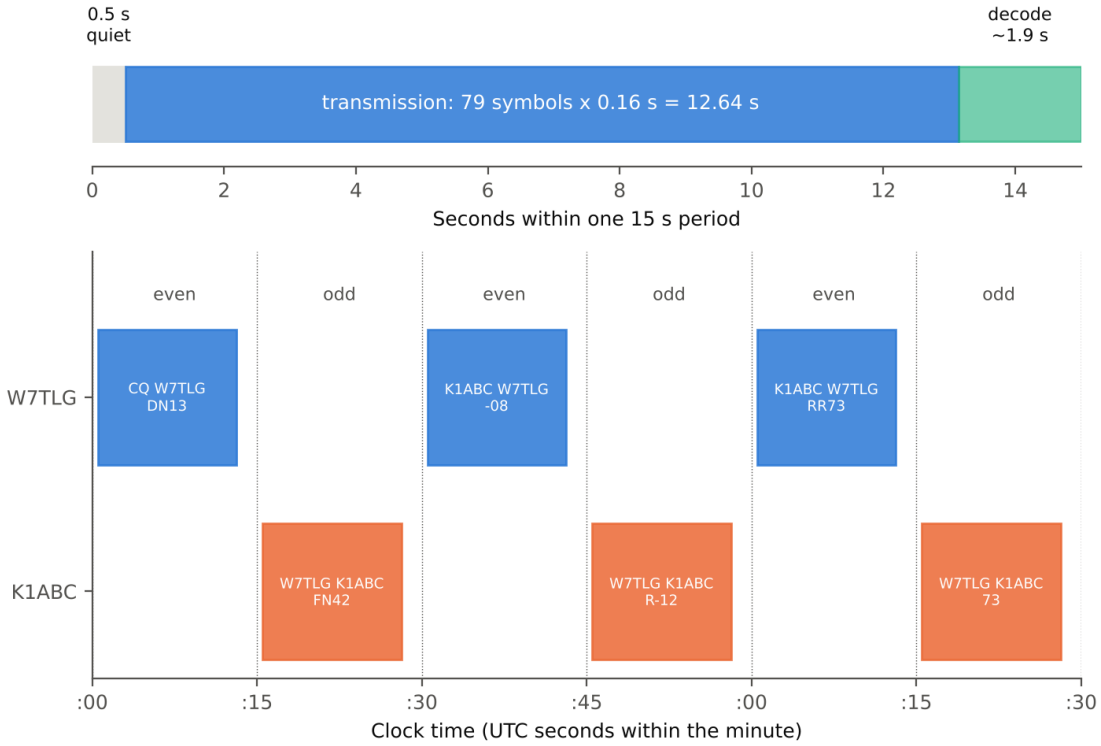


Figure 6. Top: the layout of one 15-second period. Bottom: a complete FT8 contact between W7TLG and K1ABC, six periods (90 seconds) long. The stations alternate, each transmitting in its own periods (even or odd) and listening in the other.

4.2 A complete contact

The standard exchange is deliberately short. In the example above, one station calls CQ and the other answers with its grid locator. The caller sends a signal report; the answering station sends a report back with an "R" meaning "received." The caller sends RR73, and the answering station sends 73. That is six messages, six periods, 90 seconds. Software will sequence this for you automatically, but every one of those messages is a complete 77-bit packet that has to be successfully decoded on the other end. If one fails, the sender repeats it.

4.3 Why the clock matters

Every receiver assumes the transmission begins about 0.5 s into the period. A correct decode requires that the receiver's idea of "start" be within a small fraction of a second of the transmitter's. The software searches over a range of start times around the nominal one, using the Costas arrays to find the true start (Section 8), so modest clock errors are tolerated. The offset it finds is displayed in the **DT** column of the decode list. A DT near zero means the sender's clock agrees with yours. A DT consistently different from zero for *everyone* means *your* clock is off. A DT that is persistently large for a single station means that station's clock is off, which wastes some of that station's signal. Practical advice for keeping your computer clock accurate by network time synchronization is in Section 10.

4.4 Why 15 seconds?

The length of a transmission is a trade between speed and sensitivity. For a fixed power, a longer transmission delivers more total energy to the receiver, and energy per bit is what determines whether a bit can be recovered in noise. FT8's predecessors worked on this principle in a more extreme form: JT65 and JT9 use one-minute transmissions. FT8 gives up some of that sensitivity in exchange for contacts that take a minute and a half instead of several minutes. FT4, designed for contests, goes further, with 7.5 s periods and a further loss in sensitivity.

Going deeper: energy, time and sensitivity. In white noise of one-sided power spectral density N_0 (watts per hertz), the thing that determines how reliably a modulation can be detected is the ratio of received energy to N_0. If a signal of average power S lasts for time T_{tx}, it delivers energy $S\,T_{\text{tx}}$. For a message of K information bits, the energy per bit is $E_b = S\,T_{\text{tx}}/K$. Reliable communication requires E_b/N_0 to exceed a threshold set by the modulation and code. So for the same message length K and the same threshold, doubling the transmission time halves the required power, a 3 dB gain. That is the entire reason slower modes are more sensitive: they are not magic, they just collect more energy per bit. FT8 collects its message over 12.64 s; a mode with a transmission nearly four times as long collects nearly four times the energy at the same power, which is worth up to about 6 dB if everything else is equal.

5. Fitting a message into 77 bits

5.1 The "13 characters" puzzle

An FT8 message is often described as "13 characters." That is the limit for *free-text* messages: 13 characters from a 42-character alphabet (blank, the digits 0 to 9, the letters A to Z, and the symbols + − . / ?). But most FT8 messages are not free text. A typical message like `K1ABC W7TLG R-07` is 16 characters long, and every FT8 message, whatever its length, occupies exactly **77 bits**. How?

The answer is that FT8 does not send the *text*. It sends a **number**, which the receiver looks up. The trick is to make the table of possible numbers just big enough to cover all realistic messages, and no bigger.

5.2 Free text: counting possibilities

Free text is the simplest case, so start there. There are 42 possible characters, and each message has 13 of them, so the number of possible free-text messages is $42^{13} \approx 1.27 \times 10^{21}$. How many bits do you need to give a distinct number to each of those? Since $2^{70} \approx 1.18 \times 10^{21}$ is slightly too small and $2^{71} \approx 2.36 \times 10^{21}$ is big enough, **71 bits** are needed. Treating the 13 characters as the digits of a base-42 number gives a value

from 0 to $42^{13} - 1$, and the number fits in 71 bits. Add 3 bits to say "this is a free-text message" and 3 bits for the message-type field (see below), and the total is the 77 bits.

There is nothing clever here: it is just a number system. Text is stored the way a calendar stores a date, as a count.

5.3 Standard messages: callsigns as index numbers

The messages that make up most traffic have a fixed shape: two callsigns and one more item, which is a grid locator, a signal report, or a closing word like RR73. The standard message therefore consists of these fields, in order:

Field	Bits	Contents
c28	28	First callsign (or CQ, DE, QRZ, CQ DX, …)
r1	1	Flag: this callsign has a /R or /P suffix
c28	28	Second callsign
r1	1	Flag: /R or /P suffix on the second callsign
R1	1	The letter "R" in "R-07" (meaning "roger")
g15	15	Grid locator, signal report, RRR, RR73, 73, or blank
i3	3	Message type (1 for the standard message)
	77	

The 28-bit callsign field can hold $2^{28} = 268{,}435{,}456$ different values, and they are divided into three zones. The lowest 2,063,592 values are special tokens: 0 is DE, 1 is QRZ, 2 is CQ, and the rest are variants of CQ such as CQ 123 or CQ DX. The next block of 4,194,304 (2^{22}) values is used for callsign *hashes*, discussed below. Everything else, 262,177,560 values, is used for ordinary callsigns.

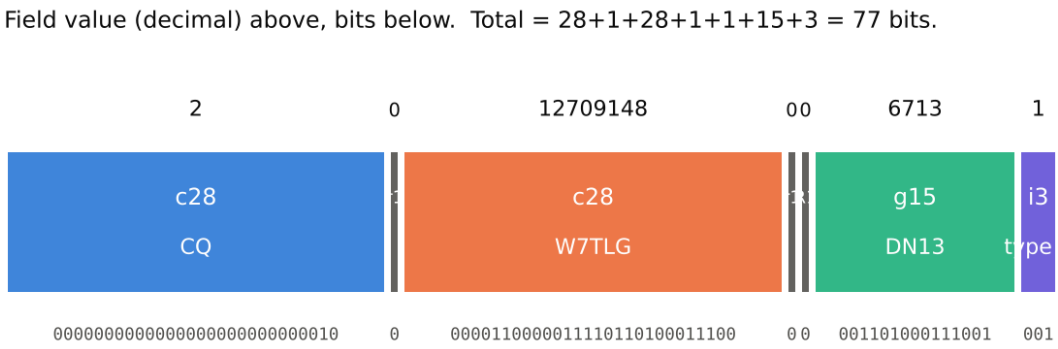


Figure 7. The 77 bits of CQ W7TLG DN13. Above each field is its value as a decimal number (CQ is token 2, W7TLG is number 12,709,148, and the grid DN13 is number 6,713); below it are the actual bits. Fields shaded gray are single flag bits, all zero here.

How does an ordinary callsign become a number? Every callsign, internationally, follows a pattern: one or two prefix characters, a digit, and a suffix of up to three letters. FT8 fits every callsign into a six-character template: the first character may be a space, a digit or a letter (37 possibilities); the second a digit or a letter

(36 possibilities); the third *must* be a digit (10); and the last three are a space or a letter (27 each). `W7TLG` has its digit in the second position, so a leading space is added (space, W, 7, T, L, G) to push the digit into the third position. The callsign is then converted to a number in the same way you would convert digits in different bases:

$$n = \Big(\big(\big((i_1 \cdot 36 + i_2) \cdot 10 + i_3\big) \cdot 27 + i_4\big) \cdot 27 + i_5\Big) \cdot 27 + i_6.$$

The total number of combinations is $37 \times 36 \times 10 \times 27^3 = 262{,}177{,}560$, which is why that is exactly the size of the block reserved for callsigns, and, remarkably, $262{,}177{,}560 + 2{,}063{,}592 + 4{,}194{,}304 = 2^{28}$ exactly. The table is full, with no room wasted. A worked example is in Appendix A.

The grid field uses the same trick. A four-character Maidenhead locator is two letters from A to R (18 possibilities each) and two digits (10 each): $18 \times 18 \times 10 \times 10 = 32{,}400$ possibilities. The 15-bit field holds $2^{15} = 32{,}768$ values, so the 368 values left over are used for the other things that can follow two callsigns: blank, `RRR`, `RR73`, `73`, and signal reports from −30 to +32 dB. For `DN13`, the number is $\big((3 \times 18 + 13) \times 10 + 1\big) \times 10 + 3 = 6{,}713$.

> **Going deeper: why this counts as compression.** Plain ASCII would need 7 or 8 bits per character: `CQ W7TLG DN13` as text is 13 characters, or 104 bits as 8-bit bytes, and `K1ABC W7TLG R-07` would take 128. The structured packing needs 77 bits for everything, a saving of roughly 25 to 40 % over plain text. The saving comes from *knowing what messages are likely*. A general text compressor cannot do this. Information theory says the minimum number of bits needed to identify one outcome out of N equally likely outcomes is $\log_2 N$. FT8 does not make all messages equally probable (it cannot send arbitrary text in a standard message), and the design accepts that restriction in exchange for brevity. Every bit saved matters because each additional bit of message needs additional bits of protection and additional seconds on the air.

5.4 Callsigns that do not fit

Not every callsign fits the six-character template: compound calls such as `PJ4/K1ABC` or long portable calls do not. FT8 handles these with two mechanisms. A non-standard call can be sent in full in a separate message type, using a 58-bit field. When it appears in a standard message, it appears as a **hash**: a 22-bit fingerprint computed from the callsign. The receiver can display the call if it has seen it in full before (it keeps a table of fingerprints), and otherwise shows a placeholder like `<...>`. This is a deliberate trade, a small chance of not being able to display a call in exchange for fitting everything into 77 bits.

5.5 Other message types

The 3-bit type field and a 3-bit sub-type allow the designers to define different layouts for different situations: free text, DXpedition (“fox and hounds”) mode, special contest exchanges, and non-standard callsigns. They all share the same shape: 77 bits with the type at the end. From the point of view of everything after the packing step (the CRC, the LDPC code, the tones, the sync), a message is just 77 bits, and the rest of the system neither knows nor cares what they mean.

6. Protecting the message: the CRC

The 77 bits are now ready to be sent, but sending them as-is would be reckless. On a noisy path, a few bits will be received incorrectly, and without some protection, a callsign could change by one character or a report could change from −12 to −13, with no way to tell.

FT8 protects the message with two layers: a checksum that detects errors (this section) and a powerful code that corrects them (the next two).

6.1 What a CRC does

A **cyclic redundancy check (CRC)** is the digital equivalent of the check digit on a credit-card number or an ISBN. The sender runs the 77 message bits through a fixed arithmetic recipe and gets a 14-bit result, which is appended to the message. The receiver runs the same recipe on what it received and compares. If even one bit changed, the two almost certainly will not match.

A 14-bit CRC has $2^{14} = 16{,}384$ possible values. If the received bits are garbage, the chance that the garbage's CRC happens to match the garbage's trailing 14 bits by accident is about 1 in 16,384, or 0.006 %. It follows that the CRC is a good filter, but not a perfect one. A decoder that tries thousands of candidates per period will occasionally accept a false one, and this is the origin of the rare nonsense decodes that every FT8 operator has seen: a plausible-looking message from a station that was never on the air.

Going deeper: the FT8 CRC. The CRC is the remainder of polynomial division over the two-element field GF(2). FT8 uses the 14-bit generator polynomial

$$g(x) = x^{14} + x^{13} + x^{10} + x^9 + x^8 + x^6 + x^4 + x^2 + x + 1,$$

written 0x6757 with the leading term or 0x2757 without it, with an initial register value of zero. It is computed over the 77 message bits followed by five zero bits (82 bits in all), and the 14-bit remainder is appended to give 91 bits. A CRC by itself *detects* errors (it catches every burst of 14 or fewer consecutive bit errors, for example); it does not say where they are or how to fix them. In FT8 its role is not to protect the message against channel errors, which is the LDPC code's job, but to tell the decoder with high confidence that the LDPC decoder has landed on the *right* codeword rather than a wrong one.

7. Low-density parity-check coding, without the pain

7.1 The basic idea: redundancy with structure

The simplest way to make a message robust is to repeat it. Send every bit three times and let the receiver take a majority vote. That works, but it is wasteful: it triples the transmission time to correct a single error per bit. A far better approach is to add redundancy cleverly, so that a *small* number of extra bits protects a *large* number of data bits, with each extra bit doing work for many data bits at once.

The cleverest simple version of this is the **parity check**. Take a few bits and add one more bit chosen so that the total number of 1s is even. If a received group has an odd number of 1s, something is wrong. One parity bit can *detect* a single error, but it cannot tell you which bit is wrong; it only says "something in this group is wrong." The remedy is to run several overlapping checks, arranged so that every bit belongs to a different combination of them. When one bit is flipped, exactly the checks that contain that bit fail and the others

pass. Since no other bit sits in precisely that combination of checks, the pattern of failures identifies the culprit. Section 7.2 shows this on a small example.

7.2 A toy example

The smallest practical example is the Hamming (7,4) code: four data bits and three parity checks over seven bits. Each check is a rule: *these four bits must contain an even number of ones.*

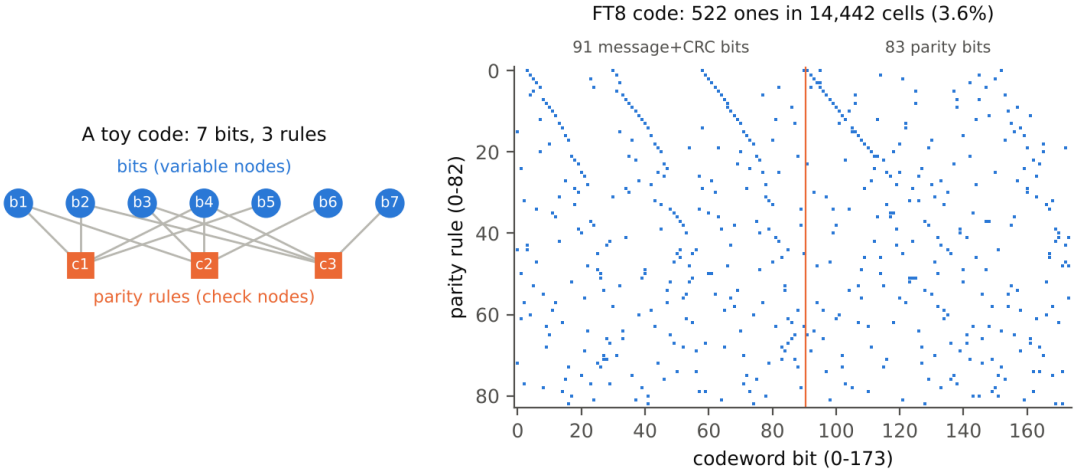


Figure 8. Left: the Hamming (7,4) code drawn as a "Tanner graph." Circles are bits; squares are parity rules; a line connects a rule to each bit it covers. Right: the same kind of picture for the real FT8 code, drawn as a matrix. Each of the 83 rows is a parity rule, each of the 174 columns is a bit, and a dot means "this bit takes part in this rule." Only 3.6 % of the cells are filled: the matrix is *sparse*, which is what "low density" means.

Suppose bit b2 is flipped in transit. The rules that include b2 (c1 and c3) fail, and the one that does not (c2) passes. The bit that is in exactly the failing rules and not in the passing one is b2, so flip it back. This works for any single error in a Hamming code.

7.3 The FT8 code

FT8's code is the same idea at a larger scale and with a modern design. It is a **(174, 91)** code: 91 information bits are expanded into a 174-bit codeword by adding 83 parity bits. Each of the 83 parity bits is the exclusive-OR of a particular subset of the 91 information bits, and the subsets were chosen by the designers (by computer search) to give the best performance. Equivalently, the codeword has to satisfy 83 parity rules, each involving six or seven of the 174 bits, and every bit takes part in exactly three rules. This is what "low density" means: each rule involves only 6 or 7 bits out of 174, and each bit takes part in only 3 rules out of 83.

The ratio 91/174 ≈ 0.52 is the **code rate**: about half of what is transmitted is information, and about half is protection. A codeword is any 174-bit pattern that satisfies all 83 rules. Out of the 2^{174} possible 174-bit patterns, only 2^{91} are legal, a vanishingly small fraction, and any two legal codewords differ in many bit positions. That distance is what lets the decoder recognize the correct codeword even after a large number of bits have been corrupted.

7.4 How decoding works: belief propagation

Low-density codes were invented by Robert Gallager at MIT in 1960 and then largely forgotten, because the computers of the day could not run the decoder. They were rediscovered in the mid-1990s, and they are now

in Wi-Fi, digital television, and 5G data channels. The reason they are so effective is a decoding method called **belief propagation**, which is easy to describe in words.

The decoder begins with a **belief** for each bit, not a hard 0 or 1, but a *confidence*: "this bit is probably a 1, with strength 2.3," or "this one is probably a 0, but only weakly." The confidence comes from the tone energies (Figure 5). For a bit that is carried by a clearly bright tone, the confidence is high. For a bit carried by an ambiguous tone, it is low. Then the following loop is repeated:

1. **Each rule looks at the beliefs of its 6 or 7 bits and tells each bit what the *others* imply.** A rule that says "these seven must have even parity" can reason: "if the other six bits are very probably 1, 1, 0, 1, 0, 0, then you, the seventh, must be a 1."
2. **Each bit combines its original evidence with what the three rules told it,** and updates its belief.
3. **Check whether all 83 rules are now satisfied.** If so, stop. If not, go around again, with each rule now working from the improved beliefs.

Weak bits are corrected by the strong bits around them. A bit that was ambiguous when received becomes confident because three independent rules, each informed by other bits, agree. The improvement cascades through the graph and typically converges in a handful of iterations when the signal is good, and in a few dozen when it is marginal.

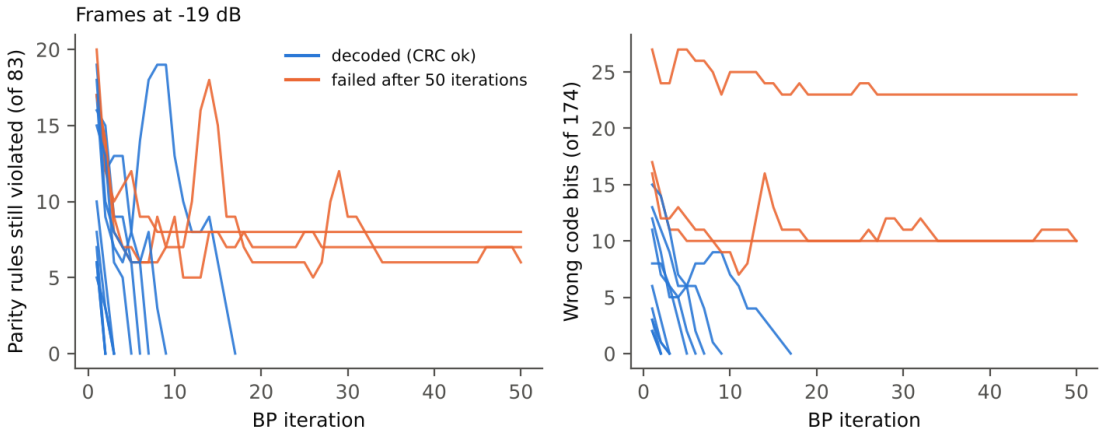


Figure 9. Belief propagation at work on 14 simulated frames at −19 dB. Left: the number of parity rules still violated, after each iteration. Right: the number of wrong code bits. Blue curves are frames that decode (the count reaches zero and the CRC then passes); orange curves are frames that fail after 50 iterations. At this SNR the raw tone measurements have roughly 10 to 30 wrong bits out of 174 before decoding. Belief propagation removes all of them in a few iterations on most frames.

When decoding succeeds, the CRC is checked as a final safeguard. When belief propagation fails, WSJT-X has a second decoder, **ordered-statistics decoding (OSD)**, which makes a more exhaustive search around the most reliable bits. The published sensitivity of WSJT-X includes this fallback. The simple receiver used for the figures in this paper uses belief propagation alone, which is why it is about 1.3 dB less sensitive (Section 9).

Going deeper: soft decisions and the sum-product algorithm. The decoder's input for each bit is the log-likelihood ratio $L_i = \ln[P(b_i = 0 \mid \text{data})/P(b_i = 1 \mid \text{data})]$. For a symbol whose eight tone amplitudes are measured, the exact likelihood of tone k, when the carrier phase is unknown, is proportional to $I_0\left(2\sqrt{E_s}\,|y_k|/N_0\right)$, where y_k is the complex output of the kth matched filter and I_0 is the modified Bessel function. The three bit LLRs of the symbol are obtained by summing likelihoods over the tones that have that bit equal to 0 and to 1. Sending bit-to-tone assignments through a Gray code (adjacent tones differ in a single bit) makes the most likely mistakes, confusing a tone with its neighbor, cost only one wrong bit rather than several.

The **sum-product** algorithm then passes messages along the edges of the Tanner graph. The message from a rule (check) j to a bit i is

$$m_{j\to i} = 2\tanh^{-1}\left(\prod_{k\in N(j)\setminus i} \tanh m_{k\to j}/2\right),$$

and the message back from a bit to a rule is the bit's original LLR plus the messages from all its *other* rules. After each iteration the posterior $L_i^{\text{post}} = L_i + \sum_j m_{j\to i}$ gives the current best guess. Appendix B works through this for the toy code by hand. The method is exact on a graph with no loops, and nearly so on sparse graphs with long loops, which is why a *low-density* code is the right choice: the sparser the graph, the better belief propagation approximates the ideal (and computationally infeasible) maximum-likelihood decoder.

8. Synchronization: the Costas array

8.1 The problem

A receiver that is listening for FT8 does not know when each signal starts, to within a fraction of a symbol; it does not know the exact audio frequency, within a few hertz; and it does not even know whether a signal is there. All three are unknown, and all three must be found *before* the demodulator can begin. And it must do so for signals that are far below the noise, so a simple "wait for a loud burst" approach does not work.

The standard solution is to include a known pattern, one the receiver can look for. If a signal is present, the receiver will find the pattern at exactly the right time and frequency; if there is no signal, it will find the pattern nowhere. The question is: what pattern is best?

8.2 What makes a good sync pattern

Suppose the pattern is a staircase: tones 0, 1, 2, 3, 4, 5, 6 in sequence. A receiver that looks for it would find a perfect match at the right time and frequency. But it would also find *partial* matches nearly everywhere nearby. Shift the staircase one symbol later and one tone higher, and it still matches in six of seven positions. A receiver cannot tell the true location from a false one that is displaced diagonally, and noise makes it even worse.

A good pattern is one that matches itself *only* when it is lined up exactly, and matches at most once when it is displaced by any amount in either time or frequency. Such a pattern was found in the 1960s by John P. Costas, for sonar and radar, where the same problem arises (is that echo at the right distance, and at the right Doppler shift?). A **Costas array** is a permutation of the tones in which no two pairs of positions have the same offset in both time and frequency. FT8 uses the 7-tone array

$$3,\ 1,\ 4,\ 0,\ 6,\ 5,\ 2.$$

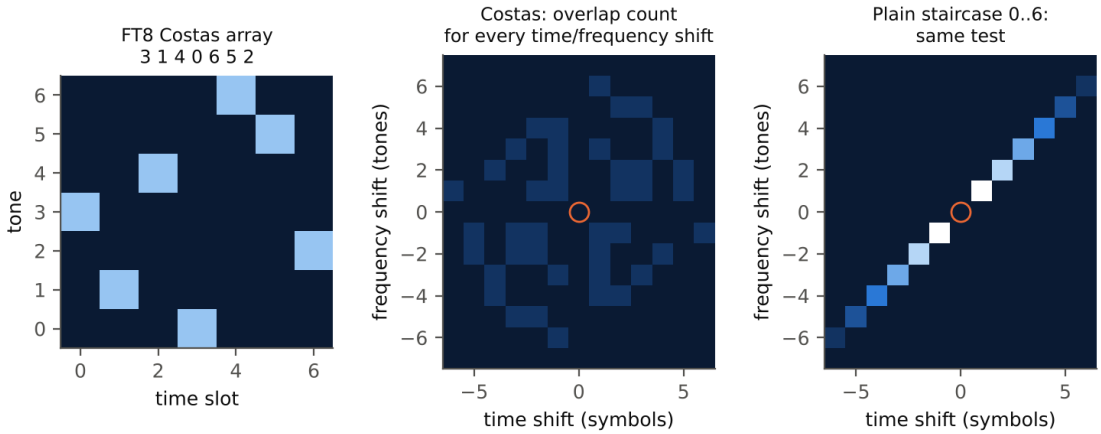


Figure 10. Left: the FT8 Costas array drawn as a grid (one filled cell per time slot). Middle: how many of its seven positions coincide with a copy of itself, for every combination of time shift and frequency shift. Apart from the exact alignment (circled), no shift produces more than one coincidence. Right: the same test for a plain staircase 0-1-2-3-4-5-6, which has up to six coincidences at diagonal offsets, so its match is ambiguous.

I checked this by computer. There are exactly 200 Costas arrays of order 7 among the 5,040 permutations of seven tones, and the FT8 array is one of them. Its worst off-peak overlap is 1; for the staircase it is 6.

8.3 Three arrays, three looks

FT8 places the same Costas array at the beginning, the middle and the end of the transmission (symbols 0 to 6, 36 to 42 and 72 to 78). This costs 21 of the 79 symbols, more than a quarter of the transmission, but the return is high:

- The receiver gets three independent looks at the signal's timing and frequency and can add them up, which makes the detection three times as strong.
- Drift or fading that changes the signal's frequency or strength during the 12.6 seconds shows up as a mismatch between the first and last blocks, so a drifting signal can be identified.
- Detecting the pattern at the beginning, middle and end of the message ensures that a candidate is the right length and in the right place, so it is very unlikely to be a coincidence.

8.4 How the search works

The receiver takes the audio and, for every possible start time (in steps of about 40 ms) and every possible frequency (in steps of about 3 Hz), it asks: "Suppose a signal began here. How much energy is at the tones the Costas array predicts, compared with the energy at the other tones?" This is a ratio score. For noise alone, the ratio is about 1/7, because one of the eight tones is as likely as another. For a real signal, the ratio is considerably higher. The candidates with the highest scores are passed to the full decoder.

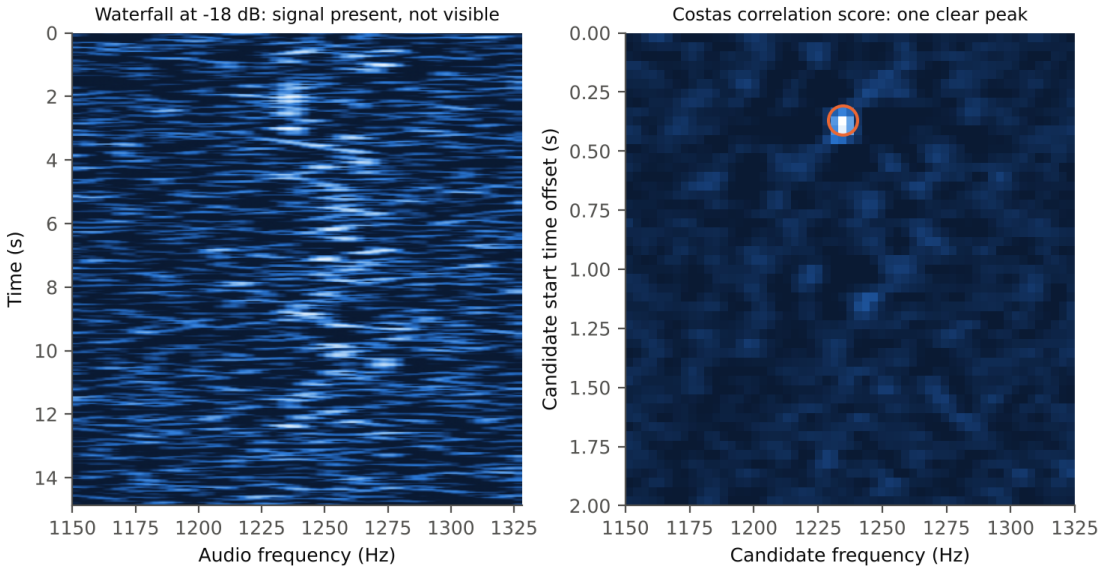


Figure 11. Finding a signal you cannot see. Left: the waterfall of a −18 dB signal at 1234.5 Hz, starting 0.37 s into the period, in noise. Nothing recognizable is visible. Right: the Costas correlation score for every candidate start time and frequency. There is a single sharp peak at the right place (circled). In this example the peak score is 0.76; the median over the whole map is 0.14, and the next-highest peak anywhere is 0.25.

The search is not perfect. Near the sensitivity threshold, noise sometimes produces a false peak that scores higher than the real signal, or the real peak is slightly misplaced; the decoder then fails or decodes a neighboring candidate. In my simulations (Section 9) this costs the simple receiver only about 0.1 dB at the 50 % decode point compared with being handed the exact timing and frequency, so for a single signal in noise the search is nearly lossless; the real difficulty in a crowded band is choosing among many candidates.

Going deeper: the price of synchronization. The 21 sync symbols carry no message information, but they carry just as much transmitted energy as the data symbols. Compared with a hypothetical mode in which all 79 symbols carried data, that is a loss of $10\log_{10}(79/58) = 1.34$ dB. The 14-bit CRC costs another $10\log_{10}(91/77) = 0.73$ dB relative to sending the 77 bits alone. These are the prices of being able to find the signal and trust the result. A Costas array is an efficient way to pay the first: with only seven symbols, each tone appears exactly once and no displacement coincides more than once.

9. Signal-to-noise ratio, “below the noise,” and Shannon

9.1 What the number on the screen means

Every line in the WSJT-X decode list begins with a time, followed by a number such as −12 or −21. That number is the signal-to-noise ratio, in decibels, in a **reference bandwidth of 2500 Hz**. It means:

$$\text{SNR} = 10\log_{10}\frac{\text{power of the signal}}{\text{power of the noise in a 2500 Hz-wide band}}.$$

The 2500 Hz reference is a convention. It is roughly the width of an SSB voice channel, and it was chosen so that SNR figures for different digital modes can be compared with each other and with ordinary voice reception. It is *not* the bandwidth of the FT8 signal, which is 50 Hz, and it is certainly not the bandwidth in

which the FT8 decoder makes its decisions, which is 6.25 Hz. The value shown is an estimate that the software makes from the signal it just decoded, so it is good to a few decibels, not to a fraction of a decibel.

A reading of −20 dB therefore means: “if I measured the noise power in a 2500 Hz-wide band, it would be 100 times greater than the signal power.” That is true, and it is the origin of the claim that FT8 decodes signals “20 dB below the noise.” But notice what has been compared: the whole signal’s power with the noise in a band fifty times wider than the signal.

9.2 Changing the bandwidth changes the SNR

Thermal and band noise are spread evenly across frequency (they are “white” over any small range), so the noise power is proportional to the bandwidth of whatever you are looking through. Cut the bandwidth by a factor of 50, and the noise power drops by the same factor, 17 dB. Cut it to the 6.25 Hz that one FT8 tone occupies, a factor of 400, and the noise drops by 26 dB.

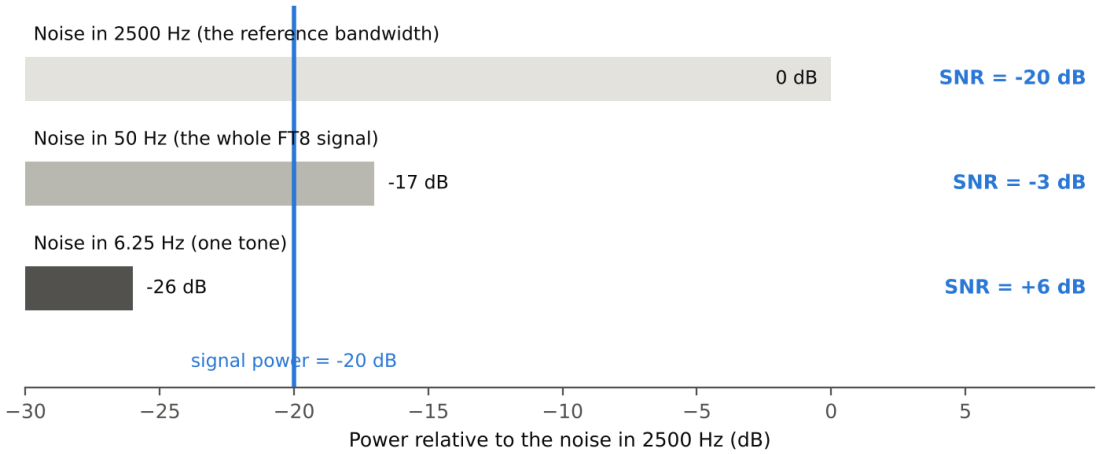


Figure 12. The same signal and noise measured in three bandwidths. The signal power is fixed at 20 dB below the noise in 2500 Hz (blue line). The noise power measured in a narrower band is lower, so the signal-to-noise ratio is higher: −3 dB across the 50 Hz that the whole signal occupies, and **+6 dB** within the 6.25 Hz that one tone occupies.

So when the SNR shown on the screen is −20 dB, the receiver’s detector, which looks at one 6.25 Hz-wide tone for one 0.16 s symbol, sees a tone that is **6 dB above the noise in its own bandwidth**. That is not a hidden signal; it is a signal 4 times as strong as the noise it competes with. It is not strong enough for a single tone to be identified reliably (in simulation, at this level roughly one tone decision in four is wrong, because among the eight candidates some noise-only tone is occasionally larger than the signal tone), but it is nowhere near “below the noise.”

The error-correcting code then does the rest. It takes 58 symbol decisions in which 20 to 30 % are wrong and recovers a message with no errors in it.

9.3 Does this violate Shannon?

No. The worry usually comes from a half-remembered idea that “information cannot be sent if the signal is below the noise.” Shannon’s theorem says something different and more useful: for a channel of bandwidth B with signal-to-noise ratio S/N, there is a maximum *rate* at which information can be sent with an arbitrarily low error probability,

$$C = B\log_2\left(1 + \frac{S}{N}\right) \text{ bits per second.}$$

When S/N is much smaller than 1, $\log_2(1+x) \approx 1.44\,x$, so the limit becomes $C \approx 1.44\,B\,(S/N)$: proportional to the SNR, and *never zero*. There is no SNR below which communication is impossible; there is only a rate that goes down as the SNR goes down. The only thing the theorem forbids is trying to send more than C bits per second.

Now put in FT8's numbers. At the published threshold of −20.8 dB in 2500 Hz, the capacity of a 2500 Hz-wide channel is

$$C = 2500\log_2(1 + 10^{-2.08}) \approx 30 \text{ bits/s.}$$

Even if you count only the 50 Hz the signal actually occupies, where the SNR is 17 dB better, the capacity is about 25 bits per second. FT8 sends 77 message bits every 15 seconds: **5.1 bits per second** of useful payload, or 7.2 bits per second if you count the CRC bits as part of what the code has to carry. FT8's rate is thus well *below* the channel capacity at this SNR, which is what the theorem requires, and decoding at −20 dB is entirely consistent with it. FT8 is slow precisely so that it can be reliable at a low SNR. A voice signal at the same SNR is hopeless because voice requires thousands of bits per second.

9.4 How close does FT8 come to the limit?

The comparison with C above is the weakest version of the question, since it is only a statement that FT8 is *possible*. The more interesting one is: **how much SNR does FT8 actually need, and how much would the best conceivable system need to send the same 91 bits in the same 12.64 seconds?** We can answer it with a short chain of calculations, which I carried out numerically (details in Appendix C).

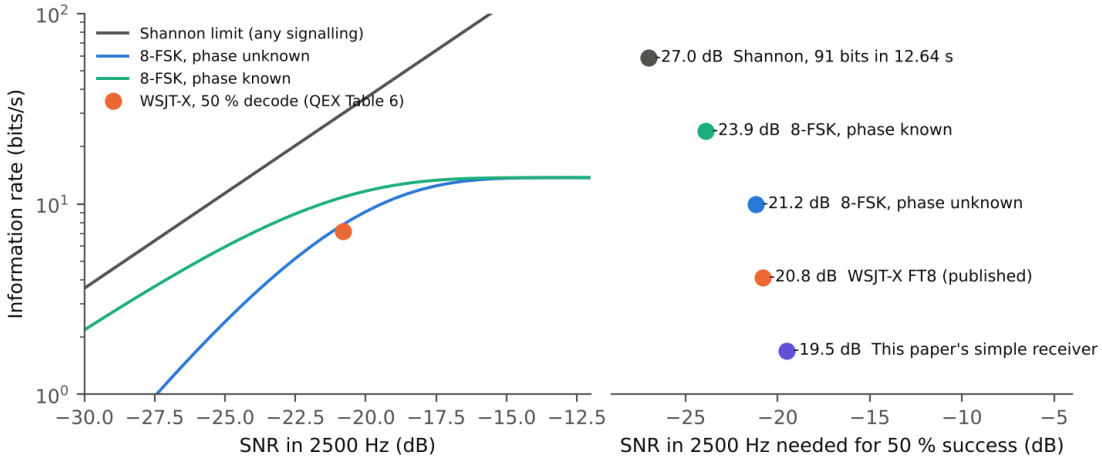


Figure 13. Left: information rate versus SNR (2500 Hz reference bandwidth). The gray line is Shannon's limit for any signaling scheme at low SNR. The blue and green curves are the limits for an 8-tone system like FT8's, when the receiver does not know the carrier phase (blue) and when it does (green); they level off at about 14 bits/s because three bits per symbol is all that eight tones can carry. The orange dot is WSJT-X's published 50 % decoding threshold for FT8 (−20.8 dB). Right: the same numbers as a ladder of required SNR.

Required SNR for 50 % success	What it represents
−27.0 dB	The absolute lower bound (Shannon) for sending 91 bits in 12.64 s with any

Required SNR for 50 % success	What it represents
	modulation and code whatever
−23.9 dB	Best possible with 8 orthogonal tones if the receiver knew the carrier phase
−21.2 dB	Best possible with 8 tones when the receiver does *not* know the phase (it does not, in FT8)
−20.8 dB	WSJT-X FT8, as published (AWGN channel, no a-priori information)
−19.5 dB	The simple belief-propagation-only receiver written for this paper

The result is remarkable. The ladder shows that WSJT-X's decoder operates within about **0.4 dB** of the theoretical limit for non-coherent 8-tone signaling. The remaining distance to Shannon's bound, about 6 dB, is not the fault of the code or the decoder. It is the price of the *modulation choice*: eight orthogonal tones, detected without knowledge of the carrier phase, are fundamentally less efficient than the theoretical ideal. Roughly 3 dB of the gap is the price of using orthogonal tones at all (rather than an ideal Gaussian-like signal), and a further 2.7 dB is the price of not tracking the carrier phase from symbol to symbol, which is why FT8 is so forgiving of drifting oscillators and unstable HF paths. The designers made that trade on purpose, in exchange for a mode that works on real HF channels with Doppler and fading.

Going deeper: how the ladder was computed. For orthogonal M-ary signaling with energy E_s per symbol in noise of density N_0, the information that one symbol conveys is the mutual information between the transmitted tone and the detector outputs. For equiprobable tones with unknown carrier phase, the likelihood of tone k given the matched-filter outputs y_j (in units where the noise has unit variance and the signal amplitude is $A = \sqrt{E_s/N_0}$) is proportional to $I_0(2A|y_k|)$, and

$$I(E_s/N_0) = \log_2 M - \mathbb{E}\left[\log_2 \sum_{k=0}^{M-1} \frac{I_0(2A|y_k|)}{I_0(2A|y_0|)}\right],$$

evaluated by Monte-Carlo (400,000 samples per point). FT8 carries 91 bits (message plus CRC) on 58 data symbols, so the code needs $I = 91/58 = 1.57$ bits per symbol, which is reached at $E_s/N_0 = 4.84$ dB without phase knowledge and 2.12 dB with it. Because each symbol lasts 0.16 s, $E_s/N_0 = 400 \times \mathrm{SNR}_{2500}$ (26.0 dB), which converts those figures to −21.2 dB and −23.9 dB. The ultimate bound uses $C_\infty = S/(N_0 \ln 2)$ for an unlimited-bandwidth Gaussian channel: delivering 91 bits in 12.64 s needs $S/N_0 \geq 4.99$ Hz, or −27.0 dB in 2500 Hz. At the 50 % point of a block code, finite block length costs almost nothing (the usual back-off scales with the standard-normal quantile of the target error rate, which is zero at 50 %), so the infinite-block-length figures are the right yardstick for this comparison. At WSJT-X's −20.8 dB, $E_s/N_0 = 5.2$ dB and $E_b/N_0 = 4.6$ dB (for 91 bits) or 5.3 dB (for 77). WSJT-X also combines two or three adjacent symbols before deciding, which in principle lets it do slightly better than the single-symbol non-coherent figure used here.

9.5 Measured decoding probability

Reaching the threshold in theory is one thing; the question for an operator is what happens in practice as the signal fades. I ran the simple receiver on 250 noisy frames at each of twelve SNR values (random 77-bit messages, random frequencies between 800 and 2500 Hz, additive white Gaussian noise, SNR defined in 2500 Hz). One receiver was told the exact start time and frequency; the other had to find them with the Costas search.

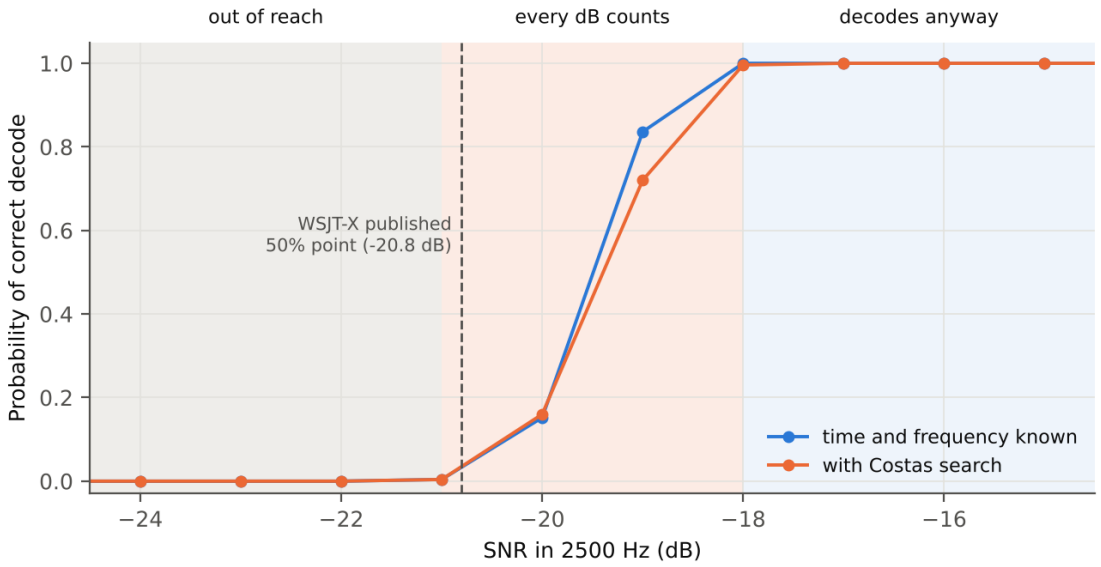


Figure 14. Probability of a correct decode versus SNR for the simple belief-propagation receiver. The transition from "always" to "never" takes only about 3 dB: all 250 frames decode at −18 dB, 72 to 84 % at −19 dB, about 15 % at −20 dB, and essentially none at −21 dB. The dashed line marks WSJT-X's published 50 % point, 1.3 dB to the left of this receiver's, thanks to the extra decoding methods in WSJT-X. The shading marks the three operating regions discussed in Section 9.6.

Two features of this curve matter for operating. First, it is a **cliff**, not a slope. A strong code decodes perfectly until it suddenly does not: between −18 and −21 dB the probability falls from 100 % to under 1 %. That is why a signal with a reported SNR of −17 dB will almost always decode and one at −22 dB almost never will, and why FT8 contacts seem to be either perfect or impossible. Second, in all 6,000 frames there was not a single false decode: whenever a message passed the CRC, it was the right message. The CRC plus the code's strength make a wrong answer very rare, but, as Section 6 explained, not impossible.

The published numbers for WSJT-X also show how much the *channel* matters. The 50 % thresholds in the QEX paper, without a-priori information, are −20.8 dB for the idealized steady channel, −18.6 dB for a mid-latitude path with 1 Hz of fading spread, and −8.6 dB for a high-latitude path with 10 Hz of spread. Rapid fading, of the kind seen on auroral and polar paths, can cost more than 10 dB, so not every weak-signal decode on a real path will reach the ideal −20 dB.

9.6 Why a few decibels matter, and where they do not

The cliff in Figure 14 has a practical consequence that is easy to miss: **a few decibels make an enormous difference, but only inside a narrow window.**

In the simulations, every frame decoded at −18 dB. One decibel lower, at −19 dB, roughly three out of four did. At −20 dB about one in seven did, and at −21 dB essentially none. A change of two or three decibels took the outcome from certain to hopeless, and a single decibel near −20 dB turns a long shot into a probable decode. The shading in the figure divides the SNR axis into three regions:

- **Above about −18 dB, the signal decodes anyway.** More signal does not make the exchange any more complete. It buys margin against fading and interference, but it does not change whether this message gets through.
- **Between about −21 and −18 dB, every decibel counts.** This is where a better antenna, a quieter receiving location, or twice the transmit power decides between a completed QSO and a missed one.

- **Below about –21 dB, the signal is out of reach.** A decibel or two will not rescue it.

Because SNR is a ratio, a decibel gained on either side of the link is worth exactly the same. On the transmit side, doubling your power (100 W to 200 W) adds 3 dB at the other station, and going from 100 W to 1 kW adds 10 dB. On the receive side, reducing the noise by 3 dB is equivalent to the other station doubling its power. There is a catch on the receive side. What counts is the noise that arrives at the receiver, and on the lower HF bands that is usually atmospheric or man-made noise coming in through the antenna, not noise generated by the receiver's own electronics. A receiver with a better noise figure does nothing there; a quieter location, a better antenna, or removing a local noise source does. On the upper HF bands at a quiet site, the receiver's own noise can matter again.

Two cautions apply. First, these numbers are from a steady, noise-only channel. Fading and multipath on real paths spread the cliff over a wider range of SNR, so the transition is less abrupt than Figure 14 suggests (the QEX thresholds in Section 9.5 show how much the channel matters). Second, the noise at *each end* of the path is different, so the link is not symmetric. You can often decode a station that cannot decode you, because its noise is higher than yours, and the reverse is equally possible.

10. Practical consequences for the operator

The theory above has a number of direct practical implications. None of them is exotic; they are the habits that distinguish a clean FT8 signal from a nuisance.

Keep your clock right. The 15-second periods are defined by UTC, and your computer is the clock. Use a network time-synchronization service (the operating system's own, or one of the freely available NTP clients) rather than setting the clock by hand, and watch the DT column: if DT is consistently large and of the same sign for most stations you decode, your clock, not theirs, is wrong. The decoder tolerates modest errors because the Costas search finds the true start, but a signal that starts far from the nominal time may have its beginning or end cut off by the boundaries of the recorded period, and the lost symbols have to be repaired by the code.

Do not overdrive the audio. The FT8 signal is a single constant-amplitude tone at any moment, so *more audio drive does not make the signal stronger*: it only makes the SSB transmitter's output stage work harder. Overdriving distorts the transmitted signal, creating spurious products outside the 50 Hz slot, where they land on other people's signals, and, on radios with ALC, makes the ALC clamp the output in ways that distort it further. A good FT8 setup shows little or no ALC activity. To get more power on the air, raise the transmitter's output power setting, not the audio level. This is one place where the "constant envelope is forgiving" idea is easily misunderstood: constant-envelope signals are *tolerant* of amplifier nonlinearity, but only up to the point that the audio is not being clipped.

Mind the duty cycle. You transmit in only one of the two alternating 15-second periods and listen in the other, so in each 30-second cycle the transmitter is keyed for 12.64 s, a duty cycle of 12.64/30 = 42 %, with about 17 s of rest. That is much lighter than transmitting continuously, but it is still heavier than voice, and each burst is a full-power, constant-amplitude signal for 12.6 s with none of the pauses that speech provides. Many transceivers and amplifiers are rated for continuous output at well below their peak SSB rating. Check the equipment's continuous-duty specification and run at a power level that it can sustain for a 12-second key-down without overheating.

Leave room. The 50 Hz width and the way signals are laid out in the passband mean that two stations transmitting on top of each other will lose some symbols, though not all of them. The error-correcting code

can repair partial collisions, and that is why many overlapping signals still decode, but each collision uses up some of the margin that was reserved for noise. Choose a frequency that has no neighbor within about 50 Hz if you can.

Interpret SNR reports with care. A report is a few-decibel estimate made by the other station's software from signals that were barely above the decoding threshold; it is valid for comparing paths on the same band and time of day, not for the precision implied by the number. It also says nothing about *your* station's absolute performance. A −18 dB report from a station in the next state with a noisy receiver and a −5 dB report from the other side of the world with a quiet one have different meanings.

Spend your decibels where they count. Your decode reports tell you where you sit on the cliff (Section 9.6). If the stations you work report you at −5 to −15 dB, more power or a better antenna will not improve those contacts. If your reports cluster around −18 to −22 dB, you are on the cliff and a few decibels will matter. For the extra margin to help, it has to come from the right place: transmit power or antenna gain for being heard, and a quieter location or antenna for hearing.

A strong signal does not block the decoder, but it can block the receiver. The decoder processes the audio mathematically, so a strong FT8 signal on the next frequency does not prevent it from decoding a weaker one. But a very strong signal in the passband can overload the receiver's front end or its analog-to-digital converter, producing distortion that appears as noise everywhere. If decoding degrades when a local station is on the air, reduce the receiver's RF gain or attenuate its input.

Expect the occasional false decode. The CRC is a 14-bit check, and any system that examines thousands of candidates per period will occasionally find one that passes by accident (Section 6). Treat an unexpected, unreproduced decode, particularly one from a rare location with an odd callsign, with healthy skepticism.

A priori decoding trades certainty for sensitivity. During a QSO, the software knows your callsign and the other station's, so it can pretend to have already decoded 62 of the 77 bits (29 for each callsign, 1 for the R flag, and 3 for the type) and search only for the remaining 15. This gains about 2 dB on a steady channel (the published threshold goes from −20.8 to −22.7 dB) and up to about 10 dB on the most disturbed channel in the QEX study, at the cost of an increased chance of a false decode. It helps a QSO already in progress; it cannot help you find a new station.

11. Summary

FT8 is a succession of small, sound decisions, each of which has a clear reason.

- **Seventy-seven bits** are enough because callsigns, grids and reports are sent as index numbers, not text. The structure of the message is the compression.
- **A 14-bit CRC** tells the receiver whether the decoder's answer is right, though it is not by itself a guarantee.
- **An LDPC (174,91) code** spends 83 parity bits to protect 91 bits, and its sparse structure lets belief propagation recover a message from tone decisions that are 20 to 30 % wrong.
- **Eight slow tones, 6.25 Hz apart,** put all the transmitted power into a sliver of spectrum, so the receiver can use a matched filter only 6.25 Hz wide and the SNR it works with is 26 dB better than the number on the screen.
- **Three Costas arrays** let the receiver find a signal in time and frequency that it cannot see on the waterfall.

- **A few decibels matter, but only near the edge.** Between about −21 and −18 dB, one or two decibels turn a miss into a decode; well above that, extra signal changes nothing, and well below it, nothing practical helps.
- **A 15-second period** trades some of the sensitivity of the slower predecessor modes for a contact that takes 90 seconds, rather than several minutes.
- **The "–20 dB" is not a paradox.** It is 6 dB *above* the noise in one tone's bandwidth; it is a rate (5 bits per second) well below Shannon's capacity at that SNR; and the design operates within about half a decibel of the best that any non-coherent 8-tone system could do.

What makes FT8 notable is less any single technique than that all of them are in balance: the modulation, the code, the synchronization pattern, the message format and the timing were designed together, and nothing is wasted.

Appendix A. A message, bit by bit

This appendix follows `CQ W7TLG DN13` from text to tones. Every number was produced by the Python encoder and matches the independent reference implementation.

Step 1: the fields.

Field	Text	Value (decimal)	Bits
c28	CQ	2	`00000000000000000000000010`
r1		0	`0`
c28	W7TLG	12,709,148	`00001100000111101101000111 00`
r1		0	`0`
R1		0	`0`
g15	DN13	6,713	`001101000111001`
i3	standard	1	`001`

The value for `W7TLG` is built from the six characters space, W, 7, T, L, G: space is character 0 of the 37-symbol first alphabet, `W` is character 32 of the 36-symbol second alphabet (digits first, then letters), `7` is 7, and `T`, `L`, `G` are letters 20, 12 and 7 of the 27-symbol alphabet (space, then A to Z). Then

$$n = \Big(\big(\big((0 \cdot 36 + 32) \cdot 10 + 7\big) \cdot 27 + 20\big) \cdot 27 + 12\Big) \cdot 27 + 7 = 6{,}451{,}252,$$

and the field value is $n + 2{,}063{,}592 + 4{,}194{,}304 = 12{,}709{,}148$. For `DN13`, $D = 3$ and $N = 13$ (counting A as 0), so $\big((3 \cdot 18 + 13) \cdot 10 + 1\big) \cdot 10 + 3 = 6{,}713$.

Step 2: the 77 bits. Concatenated:

`00000000000000000000000001000000110000011110110100011100000011010001110010 01`

Step 3: the CRC. The 77 bits are extended with five zeros and divided by the CRC polynomial. The 14-bit remainder is `01000111011001` (decimal 4,569). The 91 bits are the 77 plus these 14.

Step 4: the parity bits. The 83 LDPC parity bits are the parity-check sums of the 91 bits with the rows of the code's generator matrix. For this message they are

`10011010100011011111000100000101101011100011101011001010110110101010001100100100000000`

and the 174-bit codeword is the 91 bits followed by these 83.

Step 5: bits to tones. The 174 bits are read in groups of three. A Gray code maps the group to a tone number so that neighboring tones differ in only one bit:

Bits	000	001	010	011	100	101	110	111
Tone	0	1	3	2	5	6	4	7

For example, symbols 15 to 20 of this message carry the groups 001, 000, 000, 110, 000, 011, which become tones 1, 0, 0, 4, 0, 2. The first 25 message bits are zero (because CQ is token 2), so the transmission begins with a long run of tone 0, visible at the start of the first data block in Figure 2.

Step 6: the 79 tones. With the Costas array 3140652 at symbols 0 to 6, 36 to 42 and 72 to 78, and the 58 data tones in the two stretches between them, the complete transmission is

`3140652 00000000100402445250231433526 3140652 54235224101237073436663521100 3140652`

(spaces added for readability; the actual sequence is the 79 digits run together). The first data tones are a long run of zeros because the first 25 bits of the message are zero.

A reply. A station answering with a report sends, for example, `W7TLG K1ABC -12`. Its fields are $c28_a = 12{,}709{,}148$ (W7TLG), $c28_b = 10{,}214{,}965$ (K1ABC), and the 15-bit field $32{,}400 + 35 - 12 = 32{,}423$ (the report −12). When the other station answers `K1ABC W7TLG R-07`, the R flag is set and the field is $32{,}400 + 35 - 7 = 32{,}428$; RR73 is 32,403, and 73 is 32,404.

Appendix B. Belief propagation by hand

This is the smallest example that shows the mechanism: the Hamming (7,4) code of Figure 8, with three parity rules.

$$\text{c1: } b_1 \oplus b_2 \oplus b_4 \oplus b_5 = 0, \qquad \text{c2: } b_1 \oplus b_3 \oplus b_4 \oplus b_6 = 0, \qquad \text{c3: } b_2 \oplus b_3 \oplus b_4 \oplus b_7 = 0.$$

The message 1011 gives the codeword 1011010 (so $b_5 = 0$, $b_6 = 1$, $b_7 = 0$). Suppose the demodulator reports these log-likelihood ratios, where a positive number favors 0 and a negative one favors 1, and a larger magnitude means greater confidence:

	b1	b2	b3	b4	b5	b6	b7
Received LLR	−2.1	**−0.6**	−1.5	−2.4	+1.2	−0.9	+1.8
Hard decision	1	**1**	1	1	0	1	0
Truth	1	**0**	1	1	0	1	0

Bit b2 is wrong, but only weakly so (−0.6). The hard decisions fail rules c1 and c3 (each sums to 1) and satisfy c2. Now one iteration of belief propagation:

- **Rule c1 informs b2.** It combines the beliefs of the *other* three bits in the rule: $2\tanh^{-1}[\tanh(-2.1/2)\tanh(-2.4/2)\tanh(1.2/2)] = +0.73$. The other bits say the parity requires b2 to be a 0, with strength 0.73.
- **Rule c3 informs b2** in the same way, from b3, b4 and b7: $+0.80$.
- **b2 combines the evidence:** $-0.6 + 0.73 + 0.80 = +0.93$. The sign has flipped. b2 is now believed to be 0, which is correct.

The posterior beliefs after the first iteration are −2.29, **+0.93**, −1.71, −2.31, +0.82, −1.78, +1.49 (the hard decisions are now 1011010, all three rules are satisfied, and decoding stops). Rule c2, which does not contain b2, contributes nothing to it; the other bits' beliefs also grow stronger in the same pass because each is reinforced by its own rules.

Real FT8 decoding does exactly this on a graph 25 times larger (174 bits and 83 rules), with up to 50 iterations. Figure 9 shows how the count of violated rules typically falls.

Appendix C. The link-budget arithmetic

From SNR to energy per symbol. If the SNR in 2500 Hz is ρ (as a ratio, not in dB), the signal power is $S = \rho\, N_0\, 2500$ for noise density N_0. A symbol lasts $T = 0.16$ s, so the energy per symbol is $E_s = ST = 400\,\rho\, N_0$:

$$\frac{E_s}{N_0} = 400\,\rho \quad \Leftrightarrow \quad \frac{E_s}{N_0}\Big|_{\text{dB}} = \text{SNR}_{2500}\Big|_{\text{dB}} + 26.0.$$

At −20 dB, $E_s/N_0 = 6.0$ dB; at −20.8 dB, 5.2 dB.

Energy per message bit. Over the whole 12.64 s transmission the energy delivered is $S \cdot 12.64$ and the number of information bits is 77 (or 91, counting the CRC):

$$\frac{E_b}{N_0} = \rho \cdot 2500 \cdot \frac{12.64}{K}.$$

At −20.8 dB this is 5.3 dB for $K = 77$ and 4.6 dB for $K = 91$. The ultimate Shannon limit on E_b/N_0 for any code and any modulation is $\ln 2 = -1.59$ dB, so FT8's *per-bit* efficiency is about 6 to 7 dB from the ultimate limit, in agreement with the ladder in Section 9.4.

Shannon–Hartley at the threshold. For $\rho = 10^{-2.08}$:

$$C_{2500} = 2500\log_2(1+\rho) = 29.9 \text{ bit/s}, \qquad C_{50} = 50\log_2(1+50\rho) = 25.1 \text{ bit/s}.$$

The FT8 payload rate is $77/15 = 5.1$ bit/s, or $91/12.64 = 7.2$ bit/s counting the CRC.

Where the loss goes, in decibels (91 bits, 12.64 s).

Step	Required SNR
Ultimate limit (Gaussian signaling, unlimited bandwidth)	−27.0 dB
Use 8 orthogonal tones, receiver knows the phase	−23.9 dB
Receiver does not know the phase (non-coherent detection)	−21.2 dB
WSJT-X (published, AWGN)	−20.8 dB
This paper's belief-propagation-only receiver, timing known	−19.5 dB

Step	Required SNR
… with Costas search for timing and frequency	−19.4 dB

Absolute levels. The thermal noise floor of a perfect receiver at room temperature is -174 dBm/Hz. Across 2500 Hz that is $-174 + 10\log_{10}2500 = -140$ dBm. A signal 20 dB below that is at -160 dBm, about 10^{-19} W. On the actual HF bands the noise is mostly atmospheric and man-made rather than thermal, and is typically many decibels above the thermal floor, so the real signal power needed for decoding is larger by the same amount. The "−20 dB" is relative to *whatever noise there is*.

Appendix D. Glossary

8-GFSK. Eight-tone frequency-shift keying with Gaussian smoothing of the frequency transitions.

AWGN. Additive white Gaussian noise: the idealized noise model used for performance comparisons.

Belief propagation. The iterative message-passing algorithm used to decode LDPC codes.

Codeword. A bit pattern that satisfies all of a code's parity rules. For FT8, a legal 174-bit pattern.

Costas array. A permutation of tones in which no time-and-frequency displacement lines up more than one position with itself; used for synchronization.

CRC (cyclic redundancy check). A short checksum, 14 bits in FT8, used to detect whether decoding has produced the correct message.

DT. The time offset, in seconds, of a decoded signal relative to the expected start of the transmission.

E_s/N_0, E_b/N_0. Energy per symbol or per bit divided by the noise power spectral density.

FEC (forward error correction). Redundancy added to a message so the receiver can repair errors without a retransmission.

Gray code. An assignment of bit patterns to tones such that neighboring tones differ in exactly one bit.

Hash (callsign). A short numerical fingerprint of a callsign used in place of the full call.

LDPC (low-density parity-check) code. An error-correcting code defined by a sparse set of parity rules; FT8 uses a (174,91) code.

LLR (log-likelihood ratio). A signed number expressing how strongly the evidence favors a bit being 0 or 1.

OSD (ordered-statistics decoding). A more exhaustive decoder that WSJT-X applies when belief propagation fails.

Reference bandwidth. The 2500 Hz bandwidth in which FT8 SNR figures are expressed.

Symbol. One 0.16 s interval carrying one of eight tones, hence three bits.

Tanner graph. The bipartite picture of a code in which bits and parity rules are nodes and a line joins each rule to the bits it covers.

Appendix E. How the figures were made

The encoder (`ft8.py`) implements message packing, the CRC, the (174,91) LDPC encoder (using the code's published generator matrix), the Gray mapping, the Costas insertion and the Gaussian-filtered waveform. The receiver (`ft8dec.py`) implements a Costas-correlation search, per-symbol 8-tone detection, soft-decision demapping, sum-product belief propagation and the CRC check. `sim_snr.py` runs the Monte-Carlo decoding experiment (250 frames per SNR per receiver), `capacity.py` evaluates the non-coherent and coherent 8-tone capacity, and `figs_a.py` and `figs_b.py` produce the figures.

Validation. For thirteen test messages, the 79-tone output of the Python encoder was identical to that of the C implementation `ft8_lib`; the synthesized waveform correlated with the reference waveform at 0.999999; and the reference decoder decoded the Python-generated audio. *Limits.* All noise is white Gaussian; there is no fading, drift or interference. The simple receiver is not WSJT-X: it omits OSD, multi-symbol detection, a-priori decoding and signal subtraction. SNR is defined as signal power during the transmission divided by noise power in 2500 Hz.

References

1. S. Franke, B. Somerville and J. Taylor, "The FT4 and FT8 Communication Protocols," *QEX*, July/August 2020, pp. 7-17.
2. WSJT-X User Guide, Princeton University (K1JT), https://wsjt.sourceforge.io/wsjtx.html.
3. `ft8_lib`, an open-source C implementation of FT8 and FT4, https://github.com/kgoba/ft8_lib.
4. C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal* 27, 1948.
5. R. G. Gallager, "Low-Density Parity-Check Codes," *IRE Transactions on Information Theory* 8, 1962.
6. D. J. C. MacKay and R. M. Neal, "Near Shannon limit performance of low density parity check codes," *Electronics Letters* 32, 1996.
7. J. P. Costas, "A study of a class of detection waveforms having nearly ideal range-Doppler ambiguity properties," *Proceedings of the IEEE* 72, 1984.